Buffer First, Group Later: A Lock-Free Edge Intake for a Two-Depot Transit Fleet Monitor

Sumalatha Kambhampati
Student ID: X25156420
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25156420@student.ncirl.ie

***Abstract*—A bus depot's edge node is hit from many directions at once. Every vehicle reports several signals, each on its own thread, and all of them arrive at the same small gateway to be held until the next window closes. The obvious design, a shared buffer that every arriving reading locks, groups, and appends itself into, turns that gateway into a point of contention exactly when traffic is heaviest. This project takes the opposite approach: it buffers first and groups later. Ten sensor processes drive five bus signals, engine temperature, brake-pad wear, passenger count, fuel level, and GPS speed, across two depots; the fog node accepts each reading with a single lock-free enqueue onto a concurrent queue and does no grouping at all on the hot path, then, once per window, a single thread drains the whole queue and groups it by depot and signal in one pass before reducing each group to a summary and raising the fleet's fault alarms. Behind the edge, a serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists every window, and a dashboard served through Amazon API Gateway presents each depot as a roster of its vehicles' signals. The system was provisioned to a real Amazon Web Services account rather than a local emulator; 139 automated tests pass, and verification against the live endpoint confirmed both depots streaming all five signals, one depot raising an engine-overheat alarm while the other stayed clear, live trends, and a healthy pipeline, with the frontend delivered from Amazon S3.**

***Keywords*—public transit, fleet telemetry, intelligent transportation systems, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB**

I. Introduction

A public-transport operator running buses out of several depots needs to know the state of each vehicle continuously, not at the next scheduled inspection. An engine creeping toward overheating, a brake pad worn past its limit, or a tank about to run dry are exactly the conditions a live feed can catch and a paper round cannot, and data-driven intelligent-transportation systems have long been proposed to turn that stream of vehicle telemetry into operational decisions [1], [2]. The design question is where the stream is turned into a decision.

The Internet of Things makes it cheap to instrument every bus and depot [3], but streaming every raw sample from every vehicle to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, rarely change anything, and it defers the one moment that matters, a signal crossing into its danger range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that sits idle most of the day [8].

This project builds that two-tier design for a scaled but faithful slice of a monitored fleet: two depots, designated depot-a and depot-b, each carrying the same five signals, run as ten independent sensor processes. What gives the design its own character sits at the edge node's front door. A depot gateway is a concurrent place: readings arrive from many vehicle threads at once, and the naive way to hold them between windows, a shared map that every arriving reading locks to find its group and append itself, makes the ingest path contend for that lock precisely when the fleet is busiest. The design here refuses to group on arrival. Each reading is accepted with a single lock-free enqueue onto a concurrent queue and nothing more; all of the structure, grouping the readings by depot and signal, reducing each group to a summary, and evaluating the fault rules, is deferred to a single thread that drains the whole queue once per window. Ingest stays contention-free, and the expensive work happens once, off the hot path.

The work pursues four objectives, each demonstrated on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into an operational reading, in particular a per-depot roster of each vehicle signal and its alarms. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-depot scale rather than a city's whole fleet. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice,

including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one signal on one vehicle roster at one depot. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn from a small symmetric range and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real telemetry feed rather than independent noise. The step sizes are set per signal to match how the real quantity moves, so a passenger count that swings as a bus loads and empties drifts faster between ticks than a fuel level that falls slowly over a shift. Each process runs two independent timer threads, one for sampling and one for dispatch, so a slow dispatch, a POST waiting on a briefly unreachable gateway, can never delay the next sample; the sampler appends to a small local buffer under a short lock, and the dispatcher takes a snapshot of that buffer and clears only the readings it actually shipped, so a failed dispatch leaves everything buffered for the next attempt and nothing is lost or reordered. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal, the depot, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte, posted over HTTP to the fog node's ingest endpoint.

The fog node is a Java service built directly on the standard library's own HTTP server, running as its own container on the edge host distinct from the sensor processes and the cloud functions, with a small literal route chain rather than a web framework and a fixed pool of worker threads. Every ingest request is validated before anything is buffered, so a batch that is malformed or missing a field is rejected with a clear error rather than corrupting a window. The buffering itself is the piece of the edge that most repays attention. Because ingest requests are served concurrently on a pool of threads, the obvious buffer, a shared map from depot-and-signal to a list, would force every arriving reading to take a lock to find and extend its group, and that lock is contended exactly when traffic is heaviest. Instead the node holds an intake queue that is lock-free: each reading is wrapped as a small event and offered onto a concurrent queue with a single non-blocking operation, and no grouping, no map, and no per-group lock is touched on the request path at all. The grouping is deferred. When the window boundary arrives on a fixed schedule, one thread drains the entire queue in a single pass, bucketing the drained events by depot and signal as it goes, so all the structure is built once, off the hot path, on the thread that is about to consume it. The trade is favourable because the grouping work is not avoided, only moved: the drain is a single linear pass that runs once per window on the thread that will immediately reduce the groups, so the total cost is the same as grouping on arrival but without any of the contention. Correctness rests on one guarantee of the concurrent queue, that it preserves the order in which readings were enqueued, so within each depot-and-signal group the readings stay in arrival order and the summary's latest value is genuinely the most recent reading rather than whichever thread happened to win a lock.

When a window closes, each non-empty depot-and-signal group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the fault rules. Each rule is a small record that carries its own predicate over the summary, so the rule set reads as a flat list rather than a dispatch structure: an average engine temperature above one hundred and five degrees raises an overheat alarm, an average brake-pad wear above eighty per cent raises a service-required alarm, an average fuel level below fifteen per cent raises a low-fuel alarm, and a maximum passenger count above seventy-five raises an overcrowding alarm. That last rule reads the window's peak rather than its average on purpose, because a single overcrowded moment is a safety fact worth reporting where an average across the window would smooth it away. GPS speed deliberately carries no rule; it is windowed and shown as movement context but is not a fault in itself. The fog node also publishes this rule set, as a descriptive table naming each signal's field, operator, and limit, on a read-only endpoint, so the dashboard can show operators the thresholds in force without hard-coding them a second time.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a whole flush costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. The queue's address is resolved once and reused, retried while the queue is still being provisioned because on a fresh deployment it may not exist when the edge host boots, and a failed flush is logged while the node keeps running rather than crashing, a bias toward keeping the edge live that suits a monitoring workload where the next window is only seconds away.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's

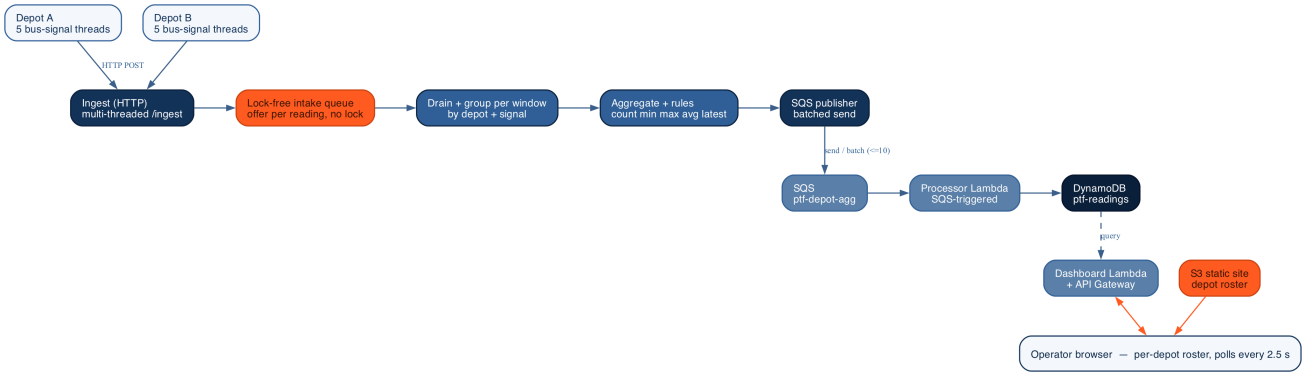

Fig. 1. Deployment topology. Ten sensor processes, two depots of five signals each, and the fog node share one Amazon EC2 edge host. On the hot path the fog node only enqueues each reading onto a lock-free intake queue; once per window a single thread drains and groups the whole queue by depot and signal, reduces each group to a summary, raises the fault alarms, and batches the results to Amazon SQS. The processor function writes each window into the Amazon DynamoDB table; the dashboard function reads it back out through Amazon API Gateway to the browser, with static assets served from Amazon S3. The dashed edge is the dashboard function's query back over the stored windows.

summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure travels back toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer's behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved in advance [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard's own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and fails a whole batch on any bad record so the queue leaves it unacknowledged and retries.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal as the partition and on a sort value that leads with the window's closing timestamp and then the depot identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the depot in the sort tail keeps the two depots' windows for the same signal from colliding.

C. Serving Layer, the Depot Roster, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Rather than write a second copy of the routing for the cloud, the read function is a thin adapter that drives the same route methods the standalone server uses, feeding each gateway request through an in-memory exchange object and reading the captured response back out, so one body of routing and query code runs both locally and in the cloud. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The function exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a depots endpoint giving the per-depot roster, a thresholds endpoint that forwards the fog node's published rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue, the processor function's deployment state, and the fog node's reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The depots endpoint is where the stored windows become an operational reading. For each signal the read function fetches the recent windows, keeps the latest per depot, and assembles a roster keyed by depot in which each depot carries a card

per signal with that signal's latest value, its window statistics, and any alarms it raised. The result reads the way an operator thinks about the fleet, one depot at a time, each depot a short roster of its vehicles' vital signs, with a raised alarm surfacing both on the offending signal's card and in a banner across the top. A fixed browser-side timer re-fetches the roster, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node's health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. Implementation

The sensor processes, the fog node, and both cloud functions are written in Java and built as independent Maven modules. The fog node and the dashboard's local server are built directly on the standard library's HTTP server with a hand-written route chain, so neither carries a web framework; the AWS SDK for Java carries the queue, table, and function calls, and the frontend renders its trends with a locally vendored charting library so the page has no external dependency at load time. The pieces are kept as small single-purpose types, the lock-free intake separated from the aggregation, the rule record separated from the publisher, and the query separated from the roster assembly, so each is unit-testable on its own without standing up a server or reaching into a live service. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 139 tests spread across every component: the bounded-walk sensor and its retain-on-failure dispatch, the ingest validation, the lock-free intake and its drain-and-group, the window aggregation arithmetic, the fault rules, the batched publisher, the processor's transform into a stored item, and, on the dashboard side, the depot-roster assembly, the pipeline health checks, the thresholds proxy, and the gateway adapter. They run without any cloud connection by exercising the units against hand-written fakes that implement the AWS SDK client interfaces, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The rule tests are pointed deliberately at the boundaries, checking a value one step inside a threshold and one step outside it, and the overcrowding rule is tested specifically to confirm it fires on a single peak that the window average leaves below the limit, which is the reason it reads the maximum rather than the mean. The publisher tests confirm that a flush is split into batches no larger than the queue's per-call limit, the processor tests check the composite sort value that orders windows within a partition, and the fog ingest path is proven with a real HTTP request against a running local server rather than only a unit test of the validator in isolation, since that request path is what the deployment actually exercises. The suite runs on every push through GitHub Actions, so a change that breaks a threshold, a batch size, a query shape, or the drain logic fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue's own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node's health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both depots rendered all five signals with their current value, and the roster read at a glance: one depot sat clear while the other raised an engine-overheat alarm as its engine-temperature average climbed past the limit, the alarm surfacing both on that signal's card and in the banner. The trend panels showed the two depots' averages moving independently across the window history, the backend-statistics endpoint showed the stored record count climbing across successive polls rather than a static number, and reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

Fig. 2. The deployed dashboard reading from the live endpoint. Each depot is a roster of its five signals as native-meter cards; a row of window-average trend charts tracks each signal across both depots; the header strip shows the four pipeline indicators. Here one depot is clear while the other raises an engine-overheat alarm, surfaced on its engine card and in the banner.

IV. Conclusion

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five signals across two bus depots into a live operational reading, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each depot-and-signal stream and raises the fault alarms locally, so only compact summaries cross the network and a fault is named in the window it appears. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 139 automated tests guarding its behaviour and continuous integration running them on every change.

The choice that shaped the system most was where to pay for structure. A depot gateway is a concurrent place, and it is tempting to organise the readings the instant they arrive, but grouping on arrival means every reading contends for the same lock at the busiest moment. Buffering first with a lock-free enqueue and grouping later, once per window on the single thread about to consume the result, kept the ingest path free of contention and moved the only real work to the one place it has to happen anyway. The supporting choice was quieter but saved real duplication: letting the cloud entry point drive the same route methods the local server uses, through an in-memory exchange, meant the routing and query logic was written and tested once rather than forked between the two deployments.

The work has clear limits. It runs at a two-depot scale with simulated sensors, its fault limits are fixed in code rather than learned from each vehicle's own baseline or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a real fleet of instrumented buses and real telemetry feeds, to move the fixed thresholds to a managed configuration an operator could tune per depot without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a vehicle's slow slide toward its limits can be trended for genuine predictive maintenance rather than only read on the latest window. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two depots to many without rethinking the tiers.

References

[1] J. Zhang, F.-Y. Wang, K. Wang, W.-H. Lin, X. Xu, and C. Chen, "Data-Driven Intelligent Transportation Systems: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 4, pp. 1624–1639, 2011.

[2] P. Killeen, B. Ding, I. Kiringa, and T. Yeap, "IoT-Based Predictive Maintenance for Fleet Management," *Procedia Computer Science*, vol. 151, pp. 607–613, 2019.

[3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.

[5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.

[10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.

[11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.

[12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.

[13] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/sqs/

[14] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/lambda/

[15] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[16] IEEE, "Manuscript Templates for Conference Proceedings," 2024. [Online]. Available: https://www.ieee.org/conferences/publishing/templates.html

The source code and deployment configuration are available at [GitHub repository URL].